# Basics Day 8 — Session 8 (Hours 29–32)

Python Programming (Basic) • Menu Loops, While Loops, Loop Patterns & Checkpoint 2

# Session 8 Overview

## Topics Covered Today

- Hour 29: Menu loops — interactive `while True` programs with state
- Hour 30: While loops, `break`, `continue`, and loop control
- Hour 31: Loop patterns — search, filter, and aggregation
- Hour 32: Synthesis, debugging, and Checkpoint 2

# Hour 29: Menu-Driven Loops and State Management

## Learning Outcomes

- Build a `while True` loop that keeps a program running until the user quits
- Display a menu and route choices using `if/elif/else`
- Explain why state (lists, dicts) must be initialized **outside** the loop
- Implement input validation to handle invalid menu choices
- Use `break` as the controlled exit from an infinite loop

## The `while True` Pattern

The three essential parts of every CLI menu:

1. **Infinite loop** — keeps the program alive
2. **Menu display** — `print()` statements showing options
3. **Input routing** — `if/elif/else` dispatching actions + `break` to quit

```
while True:
    print("\n--- Main Menu ---")
    print("1. Option 1")
    print("2. Option 2")
    print("3. Quit")

    choice = input("Enter your choice: ")

    if choice == '3':
        print("Goodbye!")
        break  # exit the loop
    else:
```

# Routing with if/elif/else

Each menu choice maps to exactly one code path:

```
if choice == '1':
    print("You chose Action 1")
elif choice == '2':
    print("You chose Action 2")
elif choice == '3':
    print("Goodbye!")
    break
else:
    print("Invalid choice. Please try again.")
```

- The `else` branch catches **any unrecognized input** — built-in validation
- Without the final `else`, invalid input is silently ignored

> Source: `Day8_Hour1_Basics.md` — Section 4 "Routing Actions"

# State Management — Data Outside the Loop

### The most common beginner mistake:

```
# WRONG — resets every iteration!
while True:
    items = []
    # ... menu logic ...
```

```
# RIGHT — persists across all menu cycles
items = []   # initialized once, before the loop

while True:
    # items survives here
    pass
```

**Rule:** Any data that must survive across menu cycles (lists, dicts, counters, flags) must be created **before** `while True:`.

> Source: `Day3_Hour1_Basics.md` — Section 4 "State Management"

# Demo: Inventory Tracker

```
inventory = []   # state OUTSIDE the loop

while True:
    print("\n--- INVENTORY MENU ---")
    print("1. Add item")
    print("2. View items")
    print("3. Search")
    print("4. Count")
    print("5. Quit")
    choice = input("\nEnter choice (1-5): ")

    if choice == '1':
        item = input("What item to add? ")
        inventory.append(item)
        print(f"Added '{item}'.")
    elif choice == '2':
        if not inventory:
            print("(empty)")
        else:
            for idx, item in enumerate(inventory, 1):
                print(f"{idx}. {item}")
    elif choice == '5':
        print("Goodbye!")
        break
    else:
        print("Invalid choice. Please enter 1-5.")
```

# Demo: Phone Book (Dict State)

Same pattern — different data structure:

```
phone_book = {}

while True:
    print("\n--- PHONE BOOK ---")
    print("1. Add  2. Look up  3. Remove  4. List  5. Quit")
    choice = input("Choose: ")

    if choice == '1':
        name = input("Name? ")
        number = input("Phone number? ")
        phone_book[name] = number
        print(f"Added {name}.")
    elif choice == '2':
        name = input("Look up: ")
        if name in phone_book:
            print(f"{name}: {phone_book[name]}")
        else:
            print("Not found.")
    elif choice == '5':
        print("Goodbye!")
        break
    else:
```

# Lab: Build a Menu Application

**Time: 30 minutes**

## Task

Build a menu-driven application using a list or dictionary of your choice. Your menu must include at least three options: **Add**, **View**, and **Quit**.

## Checkpoints

1. Initialize state outside the loop, build the skeleton, confirm it loops and exits
2. Implement "Add" — prompt for an item, append/store it, print confirmation
3. Implement "View" — iterate and display all items
4. Add one bonus feature: search, count, or delete
5. Add an `else` clause that catches invalid menu choices

## Common Pitfalls (Hour 29)

⚠️ **Data disappears** — state initialized inside the loop resets every cycle; move it before `while True:`

⚠️ **No `break`** — without a break in the Quit branch, the user is trapped forever

⚠️ **Menu only appears once** — `print` statements for the menu must be **inside** the loop

⚠️ **Crashes on invalid input** — add an `else` clause to catch unrecognized choices

⚠️ **Wrong indentation** — misaligned lines cause unexpected behavior; lines inside the loop are indented, lines outside are not

## Quick Check (Hour 29)

**Q1:** Why must data structures be initialized *before* `while True:` instead of inside the loop?

**Q2:** What happens if you forget to include `break` in the Quit branch?

**Q3:** How does the `else` clause in an `if/elif/else` chain act as input validation?

# Hour 30: While Loops and Loop Control

## Learning Outcomes

- Write and trace `while` loops with pre-test conditions
- Use `break` to exit a loop early and `continue` to skip an iteration
- Identify and prevent infinite loops
- Implement sentinel-controlled and input validation loops
- Explain when to use `while` vs. `for`

# While Loop Syntax and Flow

A `while` loop runs as long as its condition is `True`:

```
count = 0
while count < 5:
    print(f"Count is {count}")
    count += 1
print("Loop finished!")
```

**Pre-test:** the condition is checked *before* each iteration — if `False` from the start, the body never runs.

**Key difference from `for`:**

- `for` — ideal when you know how many iterations in advance (iterating over a list or range)
- `while` — ideal when the number of iterations is unknown (loop until a condition

## break and continue

`break` — exit the loop immediately, no more iterations:

```
numbers = [3, 7, 2, 9, 1, 8]
search_value = 9
index = 0

while index < len(numbers):
    if numbers[index] == search_value:
        print(f"Found {search_value} at index {index}")
        break       # stop as soon as we find it
    index += 1
```

`continue` — skip the rest of this iteration, jump to the next check:

```
count = 0
while count < 10:
    count += 1
    if count % 2 == 0:
        continue    # skip even numbers
```

# Avoiding Infinite Loops

### Infinite loop — the condition never becomes False:

```
# DANGER — count never changes, loops forever
count = 0
while count < 10:
    print(count)
    # forgot to increment!
```

```
# FIXED — increment moves toward the exit condition
count = 0
while count < 10:
    print(count)
    count += 1   # always make the condition progress
```

**Rule:** Something inside your loop must change the condition, or use `break` as your exit strategy.

> Source: `Day8_Hour3_Basics.md` — Section 5 "Infinite Loop Danger"

# Sentinel-Controlled Loop

Loop until a special "stop" value is entered — classic real-world pattern:

```
total = 0
while True:
    user_input = input("Enter a number (-1 to quit): ")
    number = int(user_input)
    if number == -1:
        break                # sentinel detected — exit
    total += number
    print(f"Running total: {total}")

print(f"Final total: {total}")
```

**Why `while True` here?** We don't know how many numbers the user will enter — only the sentinel `-1` signals the end.

> Source: `Day8_Hour2_Basics.md` — Section 5 "Sentinel-Controlled Loop"

# Input Validation Loop

Keep prompting until the user gives valid input:

```
while True:
    user_input = input("Enter a positive number: ")
    try:
        number = int(user_input)
        if number > 0:
            print(f"Great! You entered {number}")
            break           # valid — exit the loop
        else:
            print("That's not positive. Try again.")
    except ValueError:
        print("That's not a valid number. Try again.")
```

**Pattern:** loop → validate → `break` on success → re-prompt on failure

> Source: `Day8_Hour2_Basics.md` — Section 5 "Input Validation Loop"

# Lab: Grade Entry System

**Time: 20 minutes**

## Task

Build a grade-entry system that collects grades from a teacher, validates input, and reports statistics.

## Checkpoints

1. **(3 min)** Loop accepts grades 0–100, breaks on -1, rejects out-of-range with `continue`
2. **(4 min)** After the loop, calculate and print count and average
3. **(5 min)** Wrap input in `try/except ValueError` to handle non-integer input
4. **(5 min)** Add highest and lowest grade using `max()` and `min()`

## Completion Criteria

# Common Pitfalls (Hour 30)

⚠️ **Infinite loop** — forgetting to increment the loop variable or never changing the condition

⚠️ **Forgetting `break`** — validation loop accepts valid input but loops again anyway; add `break` after success

⚠️ **`continue` vs `break` confusion** — `break` exits the loop entirely; `continue` skips only the current iteration

⚠️ **`break` / `continue` outside a loop** — these keywords are only valid inside `while` or `for` loops

⚠️ **Off-by-one errors** — `<` excludes the boundary, `<=` includes it; choose deliberately

## Quick Check (Hour 30)

**Q1:** What is the key difference between a `for` loop and a `while` loop?

**Q2:** Write a `while` loop that prints the numbers 1 through 5.

**Q3:** In a sentinel loop, why do we use `while True:` instead of a specific condition?

# Hour 31: Loop Patterns — Searching, Filtering, Aggregating

## Learning Outcomes

- Implement **linear search** to find the first matching element
- Implement **search-all** to collect every matching element
- Apply **filter** patterns to create subsets of data
- Use **accumulator** patterns for totals, counts, and averages
- Combine patterns to solve multi-step data problems

# The Five Loop Patterns

| Pattern | Question Answered | Key Mechanic |
|---|---|---|
| Linear Search | "Is there a match? What is it?" | `return` on first match |
| Search-All | "What are ALL the matches?" | `append` to results list |
| Filter | "Keep only elements that qualify" | `append` to new list |
| Aggregation | "What is the total/count/average?" | accumulator variable |
| Combined | "Filter first, then aggregate" | chain the patterns |

> Source: `Day8_Hour3_Basics.md` — Section 4 "The Five Loop Patterns"

## Pattern 1 — Linear Search

Find the **first** match; stop immediately when found:

```
students = ["Alice", "Bob", "Andrew", "Carol", "Amy"]

def linear_search(lst, target):
    for item in lst:
        if item[0].lower() == target.lower():
            return item   # return immediately on first match
    return None           # not found

result = linear_search(students, "a")
print(f"Found: {result}")
```

- `return` **early** — no wasted iterations after the match
- Returns `None` if nothing matches (always handle `None` in the caller)
- Use when you need **one item** and can stop as soon as you find it

# Pattern 2 — Search-All

Find **every** match; never return early:

```
def search_all(lst, target):
    results = []            # accumulator for matches
    for item in lst:
        if item[0].lower() == target.lower():
            results.append(item)   # collect ALL matches
    return results          # return after full iteration

all_a_names = search_all(students, "a")
print(f"All A-names: {all_a_names}")
```

- **Key difference from linear search:** iterate the entire list, no early return
- Works for zero, one, or many matches
- Use when you need **all** matching items

> Source: `Day8_Hour3_Basics.md` — Demo Phase 2

## Pattern 3 — Filter

Create a **new list** containing only elements that satisfy a condition:

```
def filter_long_names(lst, min_length):
    filtered = []
    for name in lst:
        if len(name) > min_length:
            filtered.append(name)
    return filtered

long_names = filter_long_names(students, 4)
print(f"Names longer than 4 chars: {long_names}")
```

```
# Filter sales over $100
sales = [50, 150, 30, 200, 75, 110]

def filter_high_sales(sales_list, threshold):
    high_sales = []
    for sale in sales_list:
        if sale > threshold:
```

## Pattern 4 — Aggregation

Combine values into a **single summary** using an accumulator:

```
sales = [50, 150, 30, 200, 75, 110]

def sum_sales(sales_list):
    total = 0           # accumulator initialized BEFORE loop
    for sale in sales_list:
        total += sale   # update inside loop
    return total

def average_sale(sales_list):
    if not sales_list:
        return 0
    return sum_sales(sales_list) / len(sales_list)

print(f"Total revenue: ${sum_sales(sales)}")
print(f"Average sale: ${average_sale(sales):.2f}")
```

**Common accumulators:** sum starts at `0`, count starts at `0`, max starts at first item.

# Pattern 5 — Combined Patterns

Chain filter + aggregation to solve real-world questions:

```
# "What is the average of only the high-value sales?"
def avg_high_sales(sales_list, threshold):
    # Step 1: Filter
    high_sales = []
    for sale in sales_list:
        if sale > threshold:
            high_sales.append(sale)
    # Step 2: Aggregate
    if not high_sales:
        return 0
    return sum(high_sales) / len(high_sales)

print(f"Average of sales over $100: ${avg_high_sales(sales, 100):.2f}")
```

### Single-pass alternative — two accumulators:

```
high_total = 0
```

# Practice Walkthrough

Identify which pattern each block uses:

```
grades = [85, 92, 78, 88, 95, 70, 91]

# Block A — find max grade
highest = grades[0]
for grade in grades:
    if grade > highest:
        highest = grade
print(f"Highest grade: {highest}")

# Block B — keep grades >= 85
good_grades = []
for grade in grades:
    if grade >= 85:
        good_grades.append(grade)
print(f"Grades >= 85: {good_grades}")

# Block C — count passing grades
pass_count = 0
for grade in grades:
    if grade >= 70:
        pass_count += 1
```

# Lab: Grade Analysis

**Time: 30 minutes**

## Task

Apply all five patterns to a dataset of student grades and names.

## Checkpoints

1. **(4 min)** `find_student(roster, name)` — linear search, return first match or "Not found"
2. **(4 min)** `find_all_by_grade(students_dict, min_grade)` — search-all, return list of qualifying names
3. **(4 min)** `filter_positive(numbers)` — filter, return only positive numbers from a list
4. **(4 min)** `count_high_grades(grades, threshold)` — count aggregation above

# Common Pitfalls (Hour 31)

⚠️ **Accumulator inside the loop** — resets on every iteration; always initialize before `for` / `while`

⚠️ **Early return in search-all** — `return` on first match gives only one item; use `append` then return after the loop

⚠️ **Modifying the original list during filter** — use `remove()` inside `for` causes skipped items; always build a new list with `append`

⚠️ **Missing condition in aggregation** — sums ALL items; add `if condition:` before the accumulator update to filter while aggregating

⚠️ **Division by zero** — check `if len(lst) > 0:` or `if lst:` before dividing to compute averages

## Quick Check (Hour 31)

**Q1:** What is the key difference between linear search and search-all?

**Q2:** Why must an accumulator be initialized *before* the loop?

**Q3:** What happens if you don't check for an empty list before computing an average?

# Hour 32: Synthesis and Checkpoint 2

## Learning Outcomes

- Integrate variables, data structures, control flow, and I/O into one program
- Troubleshoot logic errors using `print` statements and mental tracing
- Apply defensive programming — validate inputs and guard against runtime errors
- Self-assess code against a rubric before submission
- Explain design choices (data structure selection, loop structure, error handling)

# The Three Layers of Python (Days 1–8)

```
Layer 1 — Atoms
  Variables, types (str, int, float, bool, list, dict), literals

Layer 2 — Motion
  Control flow: if/elif/else, for, while
  These decide WHEN and HOW MANY TIMES code runs

Layer 3 — Memory
  Data structures: lists and dicts that persist state
  These decide HOW YOUR CODE REMEMBERS THINGS
```

Checkpoint 2 combines all three:

- A **menu loop** (Layer 2) that reads and modifies a **task dictionary** (Layer 3)
- With **input validation** (Layer 1 + defensive coding)

> Source: `Day8_Hour4_Basics.md` — Section 4.1

# Five Common Mistakes to Catch

1. **State amnesia** — dict/list initialized *inside* the loop resets every iteration → Initialize outside `while True:`
2. **Lookup hazards** — accessing a key that doesn't exist → `KeyError` → Always `if key in dict:` before access/delete
3. **Input pollution** — `'Milk\n'` ≠ `'Milk'`; trailing whitespace breaks lookups → Always `.strip()` user input
4. **Iteration side-effects** — modifying a list while iterating causes skips → Use dict key-based deletion, or collect items to remove after the loop
5. **Scope confusion** — variable created inside `if` may not exist outside → Initialize variables at the top, update conditionally

> Source: `Day8_Hour4_Basics.md` — Section 4.2

# Debugging Methodology

### The five-step cycle:

```
1. Read the error message — file, line, error type
2. Trace backward — what input triggered it?
3. Add a debug print — expose the actual vs expected value
4. Run it — confirm diagnosis
5. Fix it — strip(), in-check, initialize outside loop
```

### Example — diagnosing a `KeyError` on delete:

```python
# Debug prints before the lookup
print(f"Looking for: '{task_name}'")
print(f"Dict keys: {tasks.keys()}")
```

You'll see instantly if `'Milk'` vs `'Milk '` is the mismatch.

> Source: `Day8_Hour4_Basics.md` — Section 5.2

# Defensive Programming Patterns

### Empty collection check:

```
if not tasks:
    print("No tasks yet!")
else:
    for task in tasks:
        print(task)
```

### Safe key lookup before modify/delete:

```
task_name = input("Which task? ").strip()
if task_name in tasks:
    del tasks[task_name]
    print("Deleted.")
else:
    print("Task not found.")
```

### Formatted task display (not raw dict):

# Lab: Checkpoint 2 — Task Management System

**Time: 30–45 minutes | Open notes, pair programming allowed**

## Requirements

1. `while True` menu loop with clean `break` on Quit
2. Menu options: **Add Task**, **List Tasks**, **Mark Complete**, **Delete Task**, **Quit**
3. Task storage as a **dict** `{'Task Name': True/False}` initialized outside the loop
4. **Add** — prompt for name, reject empty/whitespace, store with `done: False`
5. **List** — show "No tasks yet." if empty; otherwise `Name: [Done] or [To Do]`
6. **Mark Complete** — safe lookup, update to `True`; "Task not found." if missing
7. **Delete** — safe lookup, remove; "Task not found." if missing
8. **Invalid menu choice** — loops back to menu without crashing

## Completion Criteria

# Common Pitfalls (Hour 32)

⚠️ **State reset** — `tasks = {}` inside the loop wipes all data every iteration

⚠️ **`KeyError` on delete/complete** — always guard with `if task_name in tasks:` before modifying

⚠️ **Whitespace mismatch** — `'Milk\n'` and `'Milk'` are different strings; `.strip()` every `input()`

⚠️ **Raw dict output** — printing `tasks` directly shows `{'task': True}`; iterate and format instead

⚠️ **Missing task name prompt** — Mark Complete and Delete must call `input()` to get the task name before looking it up

⚠️ **No `else` for invalid menu choice** — add a final `else: print("Invalid choice, try again")` so the loop recovers gracefully

## Quick Check (Hour 32)

**Q1:** If my task list disappears after every menu cycle, where is the bug?

**Q2:** How do you safely delete a task from a dict without risking a `KeyError`?

**Q3:** What does `.strip()` do, and why does it matter for task name lookups?

**Q4:** In one sentence, what is the difference between `while True:` and `for item in tasks:`?

# Scope Guardrail — Session 8

## What We Covered (Basics Scope)

✓ `while True` menu loops with `break` and `if/elif/else` routing ✓ State management — data initialized outside the loop ✓ `while` loops, `break`, `continue`, and infinite loop prevention ✓ Sentinel-controlled and input validation loop patterns ✓ Linear search, search-all, filter, and aggregation patterns ✓ Combining patterns to process datasets ✓ Defensive programming — `.strip()`, `in` checks, empty-list guards

## Not Yet (Advanced Topics)

✗ File I/O — saving or loading data to disk ✗ JSON, databases, or web APIs ✗ Classes and object-oriented design ✗ List comprehensions and generator expressions ✗ Lambda functions and decorators ✗ Multi-file application architecture ✗ Advanced

# Session 8 Wrap-Up

## What We Built Today

### Hour 29 — Menu Loops

- Interactive `while True` programs with persistent state
- Routing choices with `if/elif/else` + `break` for controlled exit

### Hour 30 — While Loops and Loop Control

- `while` syntax, pre-test conditions, `break` and `continue`
- Infinite loop prevention, sentinel and validation patterns

### Hour 31 — Loop Patterns

- Five core patterns: linear search, search-all, filter, aggregation, combined

# Next Session Preview

## Session 9 (Hours 33–36)

- Hour 33: Defining and calling functions (`def`, parameters, return)
- Hour 34: Scope — local vs. global variables
- Hour 35: Refactoring a menu application into functions
- Hour 36: Checkpoint 3 — functions assessment

The menu loop you built today will get **refactored into clean functions**: `def add_task()`, `def list_tasks()`, `def delete_task()` — same logic, much better structure.

# Questions?

**Checkpoint 2 Reminders:**

- Initialize your data structure **outside** `while True:`
- `.strip()` every `input()` call
- Guard every modify/delete with `if key in dict:`
- Test edge cases: empty list, nonexistent task, invalid menu choice

# Thank You!

See you in Session 9 — Functions!